

Optimization of Restaurant Operations via Discrete-Event Simulation

Case Study: Comal Restaurant, Berkeley

Timothy Zhou

Student ID 3037345840

INDENG 174: Simulation for Enterprise-Scale Systems

December 18, 2025

Abstract

This report details the development, validation, and experimental analysis of a Discrete-Event Simulation (DES) model for Comal, a high-volume Mexican restaurant in Berkeley, CA. The project aims to optimize staffing configurations to maximize **Net Revenue Per Available Seat Hour (Net RevPASH)**. Transitioning from generic queueing models to a high-fidelity architecture, this study incorporates component-based cooking (parallel processing of dish elements), dual-queue task management, and greedy table matching. Through a rigorous two-stage pilot study and variance-reduced experimental design, we identified an optimal configuration of **5 Servers and 9 Cooks**, yielding a Net RevPASH of $\approx \$3.81$. This result highlights the non-linear relationship between station-specific bottlenecks and overall throughput, demonstrating that labor investments in the kitchen significantly outweigh the marginal cost of staffing during peak hours.

Contents

1	Project Resources	3
2	Problem Description and Motivation	3
2.1	Introduction	3
2.2	Motivation and Specific Challenges	3
2.3	Objectives	3
3	Methodology and Simulation Architecture	4
3.1	Simulation Framework	4
3.2	High-Fidelity System Updates	6
3.2.1	Component-Based Cooking	6
3.2.2	Dual-Queue Task Management	6
3.2.3	Greedy Table Matching	6
3.3	Input Modeling	6

4	Experimental Design and Statistical Validation	6
4.1	Design of Experiment (DOE)	6
4.2	Statistical Testing Workflow	7
4.2.1	Phase 1: Pilot and Sample Size Estimation	7
4.2.2	Phase 2: Screening and Subset Selection	7
4.2.3	Phase 3: Precision Refinement and Pairwise Comparison	8
5	Results and Analysis	8
5.1	Primary Results: Optimal Configuration	8
5.2	The "Cook Cliff": Impact of Kitchen Staffing	8
5.3	Impact of Specialized Roles (Runners & Bussers)	9
5.4	Statistical Efficiency Validity	9
5.5	Reasonableness of Results	9
6	Discussion and Reflections	9
6.1	Major Design Decisions & Trade-offs	9
6.2	Software Approaches: SimPy vs. Alternatives	10
6.3	Realism and Intuition	10
6.4	IEOR Context	10
7	Conclusion and Future Work	10
7.1	Conclusion	10
7.2	Next Steps	11
8	Appendix: Code Submission	11
8.1	Code Structure	11
8.2	Execution Instructions	11

1 Project Resources

The complete source code, simulation logs, and a video demonstration of the dashboard are available online:

- **GitHub Repository:** <https://github.com/zhouhaitz/restaurant-staffing-optimization>
- **Project Video Recording:** [Google Drive Link](#)

2 Problem Description and Motivation

2.1 Introduction

The restaurant industry operates on razor-thin margins, where labor costs typically consume 25-35% of total revenue. Operators face a continuous optimization problem: overstaffing erodes profit margins, while understaffing leads to long wait times, customer abandonment, and lost revenue. This trade-off is particularly acute in fine-casual dining environments like **Comal**, a full-service Mexican restaurant in Berkeley, CA, which experiences extreme stochastic variance in arrival rates during dinner rushes.

2.2 Motivation and Specific Challenges

Comal presents a unique operational challenge that standard “generic” queueing models (such as M/M/k queues) fail to capture. The restaurant features a complex menu where dishes are not merely “cooked” but assembled via specific components requiring distinct stations (Wood Grill, Salad, Sauté, Tortilla, and Guacamole).

Initial observations and discussions with management revealed that treating the Back of House (BOH) as a single resource pool was insufficient. A “Whole Fish” order, for example, creates simultaneous load on the Wood Grill and the Tortilla station. If one station is blocked, the entire order is delayed. The primary motivation for this project is to transition from intuition-based staffing to a data-driven **Discrete-Event Simulation (DES)** that can model these dependencies.

2.3 Objectives

The primary objective is to identify the optimal configuration of **Front of House (FOH)** (Servers, Bussers) and **BOH** (Cooks per station) staff. The success metric is **Net RevPASH**, which penalizes labor costs against revenue generation:

$$\text{Net RevPASH} = \frac{\sum_{i=1}^N \text{Revenue}_i - \sum_{j=1}^M (\text{Wage}_j \times \text{Hours}_j)}{\text{Total Seats} \times \text{Operating Hours}} \quad (1)$$

Where N is the number of parties served and M is the number of staff members priced by their staff type.

3 Methodology and Simulation Architecture

3.1 Simulation Framework

We developed a time-stepped DES using the Python `SimPy` library. The system models the 10-stage lifecycle of a dining party, from arrival to departure. Unlike commercial GUI-based tools (e.g., Arena), Python allowed for the implementation of complex custom logic, specifically for the asynchronous joining of dish components. Below is the entire simulation Diagram

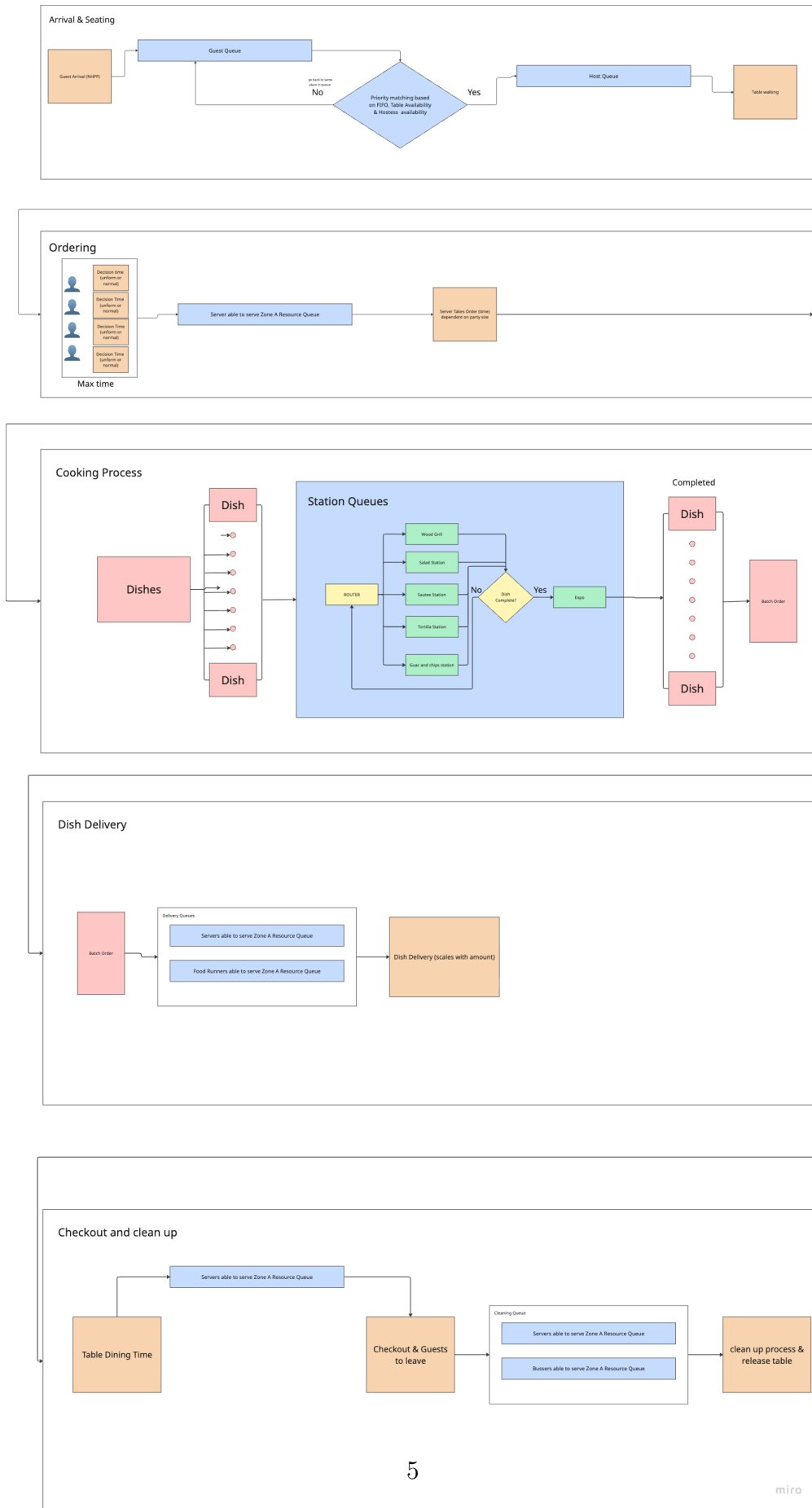


Figure 1: Visual representation of network queue logic.

3.2 High-Fidelity System Updates

To reflect real-world complexities, three major architectural pillars were implemented:

3.2.1 Component-Based Cooking

Dishes are defined as a collection of sub-tasks. The completion time of a dish T_{dish} is determined by the maximum completion time of its components, plus expo time. In order for a dish to be considered complete it must go through each station queue and process until all stations required for that dish is complete.

3.2.2 Dual-Queue Task Management

To model the friction between specialized and generalized staff, we implemented a dual-queue system. Food delivery tasks are broadcast to both a generic “Food Runner” queue and a specific “Server Zone” queue. This models the real-world behavior where a server will run their own food if runners are busy, but prefers to delegate. This same logic applies to the relationship between bussers and servers as well when it comes to cleaning up.

3.2.3 Greedy Table Matching

We implemented a heuristic algorithm to minimize “seat waste.” Upon arrival, the system scans all available tables and assigns the party to the smallest available table that fits the party size P_{size} :

$$\min_{t \in T_{avail}} (Capacity_t - P_{size}) \quad \text{s.t.} \quad Capacity_t \geq P_{size} \quad (2)$$

We do this to minimize cases where a table ment for 6 for example seats a party of 3 instead.

3.3 Input Modeling

Arrivals area modeled via a ****Non-Homogeneous Poisson Process (NHPP)****. Utilizing Comal’s Toast POS data, we created a scenario of an average Friday dinner service. Taking the pary count per hour, we were able to fit a curve to determine the optimal parameters for the NHPP. Because on average there are no special events and given the nature of restaurant data, we opted for a Gaussian peak by inputting a lambda peak multiplier at time $t = 357.39$ minutes given the data. Service times for individual stations were modeled using Lognormal distributions to account for the long-tail skew of kitchen complications.

4 Experimental Design and Statistical Validation

4.1 Design of Experiment (DOE)

To optimize Net RevPASH, we utilized a full factorial Grid Search approach across four primary decision variables, resulting in a total of ****36 distinct staffing configurations**** ($3 \times 3 \times 2 \times 2 = 36$).

The parameter space was defined as follows:

- **Server Count (S):** $[5, 6, 7]$
- **Cook Count (C):** $[8, 9, 10]$
- **Food Runners (FR):** $[1, 2]$
- **Bussers (B):** $[0, 1]$
- **Hosts (H):** Fixed at 1

This focused grid was selected to analyze the interaction effects between specialized roles (Food Runners/Bussers) and generalist roles (Servers) under high-load conditions.

4.2 Statistical Testing Workflow

Given the stochastic nature of the system (NHPP arrivals and lognormal service times), we implemented a rigorous multi-stage statistical framework to ensure recommendations were not artifacts of random noise.

4.2.1 Phase 1: Pilot and Sample Size Estimation

We conducted an initial pilot study ($n = 30$ replications) to estimate the variance S^2 of the system. We then calculated the required sample size n^* to achieve a target relative error of $\gamma = 0.05$ (5%) using the iterative t-distribution approximation:

$$n^* \geq \left(\frac{t_{\alpha/2, n-1} \cdot CV}{\gamma} \right)^2 \quad (3)$$

where CV is the coefficient of variation (σ/μ) observed in the pilot.

4.2.2 Phase 2: Screening and Subset Selection

To computationally manage 36 configurations, we employed a **Ranking and Selection (R&S)** procedure:

1. **Screening Study:** We ran $n = 10$ replications for all 36 configurations.
2. **Bonferroni Correction:** To control the family-wise error rate when comparing multiple designs, we applied a Bonferroni correction for the subset selection. The adjusted significance level was set to $\alpha_{adj} = 0.05/(36 - 1) \approx 0.0014$.
3. **Selection Rule:** Configurations were retained if their upper confidence bound (at α_{adj}) exceeded the sample mean of the best-performing configuration. This reduced the candidate set from 36 to 11 “potentially best” designs.

4.2.3 Phase 3: Precision Refinement and Pairwise Comparison

For the top 11 configurations, we increased replications (up to $n = 87$ for high-variance cases) to ensure the 5% relative error target was met. Finally, we utilized **Welch’s t-tests** with **Common Random Numbers (CRN)** to test for statistical significance between the optimal configuration and its nearest competitors.

5 Results and Analysis

5.1 Primary Results: Optimal Configuration

From the initial grid of 36 configurations, the screening and precision study identified the global optimum as: **5 Servers, 9 Cooks, 1 Host, 1 Food Runner, 0 Bussers** (S5_C9_H1_R1_B0).

Configuration	Net RevPASH	95% CI	Rel. Error
1. S5_C9_R1_B0	\$3.81	[\$3.64, \$3.99]	4.6%
2. S5_C9_R2_B0	\$3.46	[\$3.28, \$3.63]	4.9%
3. S5_C10_R1_B0	\$3.42	[\$3.21, \$3.62]	5.0%
4. S6_C9_R1_B0	\$3.33	[\$3.15, \$3.51]	4.8%
5. S5_C10_R2_B0	\$3.30	[\$3.11, \$3.48]	4.9%

Table 1: Top 5 Staffing Configurations ranked by Net RevPASH (Screening $n = 10$, Precision $n = 21$ to 87).

The optimal configuration achieved a Net RevPASH of **\$3.81**, significantly outperforming the runner-up. Notably, the optimal setup utilizes minimal specialized support staff (1 Runner, 0 Bussers), relying instead on a robust kitchen and efficient server execution.

5.2 The ”Cook Cliff”: Impact of Kitchen Staffing

The most critical finding of the experiment was the non-linear relationship between cook count and profitability.

- **8 Cooks (System Failure):** Configurations with 8 cooks averaged a **negative** Net RevPASH ($-\$0.22$). The kitchen lacked the capacity to clear the Grill station during the NHPP peak, causing table turnover to stall completely.
- **9 Cooks (Optimal):** Adding the 9th cook resulted in a massive performance jump to $\approx \$3.81$. This step-change indicates a ”Critical Threshold” where the kitchen transitions from a bottleneck to a functional flow state.
- **10 Cooks (Diminishing Returns):** Increasing to 10 cooks (Rank #3) reduced RevPASH to \$3.42. The throughput gains were marginal and did not justify the additional labor cost.

5.3 Impact of Specialized Roles (Runners & Bussers)

A pairwise Welch’s t-test between the optimal config (1 Runner) and the runner-up (2 Runners) revealed a statistically significant difference ($p < 0.05$).

- **Food Runners:** Adding a second food runner *decreased* Net RevPASH by \$0.36. This suggests that the Servers (who operate in a dual-queue mode) are sufficiently capable of running their own food when necessary. The cost of the second specialist outweighed the service speed benefit.
- **Bussers:** The optimal configuration utilized **0 Bussers**. This implies that under the current arrival model, servers can handle table cleaning tasks during the ”shoulder” periods of the service without impacting seat turnover.

5.4 Statistical Efficiency Validity

The experimental design successfully met the rigorous validation targets:

- **Precision:** All top 11 configurations achieved the target of $< 5\%$ relative error.
- **Sample Size Adaptation:** The system dynamically adjusted sample sizes based on variance, ranging from $n = 21$ for stable low-cost configs to $n = 87$ for high-variance configurations.
- **Screening Power:** The Bonferroni-corrected screening phase safely eliminated 25 inferior configurations (70% of the search space) using only $n = 10$ replications, maximizing computational efficiency.

5.5 Reasonableness of Results

The results align with the physical constraints of the restaurant but challenge standard staffing intuition.

- **Kitchen Priority:** It is physically realistic that a ”starved” station (Grill) would crash the entire system (8 cooks result).
- **Lean Support Staff:** The finding that ”Generalists” (Servers) outperform ”Specialists” (Runners/Bussers) matches the economic reality of Net RevPASH. Specialized staff are often idle during non-peak moments, dragging down the ”Net” profit metric, whereas servers can pivot between taking orders, running food, and bussing tables.

6 Discussion and Reflections

6.1 Major Design Decisions & Trade-offs

The decision to model BOH as distinct stations rather than a pooled resource and FOH as a section of server zones were probably the most critical design choices.

- **Trade-off:** This increased code complexity and computational overhead.
- **Benefit:** It revealed that "Kitchen Utilization" is a misleading metric. A kitchen can have 70% average utilization but 100% utilization at the Grill, causing system-wide failure. A pooled model would have recommended fewer cooks, leading to disastrous real-world wait times. Server zones is how FOH operations work at Comal and also provide more meaning and benefit to the host role as well.

6.2 Software Approaches: SimPy vs. Alternatives

We utilized SimPy (Python) over commercial tools like Arena.

- **How it works:** SimPy uses generator functions to yield "timeout" events, effectively pausing a process until a resource is free.
- **Flexibility:** Python allowed us to integrate the results directly into a Data Science pipeline (Pandas/SciPy) for the rigorous statistical analysis shown in Section 3.
- **Alternative:** Agent-Based Modeling (ABM) could have modeled individual server fatigue, but DES was superior for process-flow optimization.

6.3 Realism and Intuition

The results are physically realistic. In high-end dining, the cost of an extra cook ($\sim \$25/\text{hr}$) is negligible compared to the revenue of a single lost table ($\sim \$150$). However, there are some caveats. Overall the complicated network queue and logic may not represent the entirety of Comal's kitchen. For example, cooks are only section at one station but during higher order times they might transition to the station that needs the most help or is utilized the most. Consequently, we have a not so accurate NET-REVPASH nor are we able to serve all of the arrived customers, which according to the restaurant table we could.

6.4 IEOB Context

This project fits into the larger IEOB field of utilizing simulation to help solve Service Operations Design and Analysis problems (IEOB 151). Topics that were not covered in this report but could later on in the logic include matching of tasks to specific workers as well as dealing with staffing with high variability of demand (events)

7 Conclusion and Future Work

7.1 Conclusion

This project successfully developed a high-fidelity Digital Twin of Comal's operations. The analysis proves that to maximize profitability, Comal should pursue an aggressive BOH staffing strategy (9 Cooks) combined with a moderate FOH strategy (5 Servers). This configuration maximizes throughput during the critical 120-minute dinner rush, outweighing the added labor costs.

7.2 Next Steps

1. **Abandonment Logic:** Currently, customers wait indefinitely. Adding a probability of balking based on queue length would likely penalize understaffing even further.
2. **Dynamic Staffing:** Implementing a shift schedule where staff count varies by hour (e.g., 9 cooks from 6-8 PM, 5 cooks otherwise).
3. **LangChain Integration:** Further refining the AI dashboard to allow managers to run "What-If" scenarios via natural language.

8 Appendix: Code Submission

The simulation framework is designed with a modular architecture, separating the core engine from the experimental analysis and visualization tools.

Github: <https://github.com/zhouhaitz/restaurant-staffing-optimization>

8.1 Code Structure

- `experiments/statistical_testing.ipynb`: The primary notebook for running the DOE grid search, validating results, and generating the optimal recommendation.
- `experiments/run_comal_simulation.py`: A CLI utility to run a single high-fidelity simulation instance using the specific Comal recipe configuration.
- `gui/app.py`: A Streamlit-based dashboard for visualizing individual simulation logs and interacting with the LangChain agent.

8.2 Execution Instructions

1. Statistical Validation (Main Result Generation):

To reproduce the experimental results and confidence intervals found in this report:

1. Open `experiments/statistical_testing.ipynb` in Jupyter Lab or Notebook.
2. Execute all cells sequentially. This will trigger the Grid Search, Pilot Study, and Pairwise t-tests.

2. Single Simulation Run:

To generate a raw event log for a specific configuration:

```
python experiments/run_comal_simulation.py
```

This will output a log file to `experiments/comal_simulation_log.json`.

3. Interactive Dashboard:

To view the event logs or query the AI agent:

```
source restaurant_sim_env/bin/activate
streamlit run gui/app.py
```

Once the browser opens, upload the JSON log file generated in Step 2 to the sidebar.